

Prompting Fundamentals on Real Tasks v2

Name: Jiya Yadav

Code: FL-02

Track: General AI Fluency

Week: 2 Assignment A4

Voice Card

Practical, Focused, Honest, Plainspoken, Authentic, Systematic, Uncompromising

The Task

Write a GitHub README for the FlyRank Week 2 CRUD API.

This is a real task from my FL-01 audit (Task #2: Writing and updating GitHub README and documentation). I do this for every project. Quality matters and is measurable — a stranger should understand the project in under 2 minutes.

V0 — Naive Baseline

Prompt:

Write a GitHub README for my FastAPI todo CRUD API.

Output (Representative Excerpt):

FlyRank Todo API

A REST CRUD API for managing tasks, built with FastAPI as a structured backend internship assignment. The core constraint: in-memory storage only, no database.

What it does

- Full CRUD for tasks (GET, POST, PUT, DELETE)
- Query filtering on list retrieval
- /stats — aggregate counts
- /reset — wipes store back to empty

- Strict HTTP status codes: 200, 201, 204, 404, 422

“Note: Claude had no project details, so it made assumptions — added a priority field that doesn't exist in my real project, assumed a folder structure I never described, and flagged its own assumptions explicitly before generating.”

Note (V0)

What changed in the prompt: Nothing. Naive baseline.

What improved in the output: Claude produced something structurally reasonable — it included an endpoint table, a design decisions section, and a Windows note. Better than expected for a one-line prompt. However, it invented details. It added a priority field that does not exist in my project and assumed an app/ folder structure I never described. It flagged this honestly before generating, which revealed the core problem: the prompt gave it nothing real to work with, so it filled the gaps with plausible fiction.

What still failed:

- Priority field invented — not in my actual project
- Folder structure assumed, not real
- AI Rematch section invented — details do not match what actually happened
- A stranger reading this would get wrong information about my project
- Could describe any FastAPI project

What I'd try next: Assign a role so the model understands who is writing and for whom.

V1 — Role Assignment

Technique: Role Assignment

Prompt:

You are a senior backend engineer writing a GitHub README for your own project. Write a README for a FastAPI todo CRUD API. Write it as someone who built this and wants other developers to understand it quickly.

Output (Representative Excerpt):

FlyRank Todo API

A REST CRUD API for managing tasks, built with FastAPI and Pydantic — in-memory only, no database.

Design Decisions

****Why PUT accepts partial fields instead of enforcing full-resource replacement: **** strict REST says PUT replaces the whole resource and PATCH handles partial updates. I deliberately didn't add a separate PATCH endpoint —the assignment scope was about getting validation and status-code behavior right.

****Why /reset exists as an endpoint instead of just restarting the server:**** in-memory storage means state persists for the life of the process. /reset gives a deterministic starting point without killing uvicorn.

Note (V1)

What changed in the prompt: Added role: senior backend engineer writing about their own project.

What improved in the output: The tone shifted immediately from tutorial-like to developer-like. Design decisions appeared with actual reasoning — not just what was built but why alternatives were rejected. The output became opinionated. Phrases like "this is a known deviation from strict REST semantics, not an oversight" showed the model writing as someone accountable for their choices.

What still failed:

- Still invented details — same priority field appeared again
- The model had no real project information, so it still filled gaps
- Output was confident but not accurate to my project
- Role improved voice but cannot fix missing context

What I'd try next: Add real context — give the model the actual project details so it stops inventing.

V2 — Real Context

Technique: Real Context

Prompt:

You are a senior backend engineer writing a GitHub README for your own project. Write a README for this specific project:

This is a REST CRUD API built with Python and FastAPI for a structured backend internship assignment. The constraint was in-memory storage only — no database. The goal was to practice HTTP correctness, REST design, and Pydantic validation under real constraints. It includes full CRUD for tasks, partial PUT updates, query filtering, a /stats endpoint, a /reset endpoint, and strict HTTP status codes (200, 201, 204, 404, 422). Tested on Windows using curl.exe due to PowerShell JSON quoting issues.

Output (Representative Excerpt):

Tech Stack

Layer	Choice	
-----	-----	
Language	Python 3.11+	
Framework	FastAPI	
Validation	Pydantic v2	
Server	Uvicorn (ASGI)	
Storage	In-memory Python dict (no DB)	
Test tool	curl.exe (Windows)	

Endpoints

Method	Path	Status Code(s)	Description	
-----	-----	-----	-----	
POST	/tasks	201 / 422	Create a new task	
GET	/tasks	200	List all tasks	
GET	/tasks/{id}	200 / 404	Retrieve single task	
PUT	/tasks/{id}	200 / 404 / 422	Partial update	
DELETE	/tasks/{id}	204 / 404	Delete task	

GET	/stats	200	Aggregate stats	
POST	/reset	200	Clear all tasks	

Note (V2)

What changed in the prompt: Added real project details: internship context, in-memory constraint, specific endpoints, status codes, and Windows testing detail.

What improved in the output: This was the biggest single jump across all six versions. The invented priority field disappeared. The endpoint table matched my actual project. The Windows note was specific and correct. The tech stack table appeared with accurate entries. Design decisions referenced the real constraint rather than a hypothetical one. A stranger reading this would now get accurate information about the actual project — not a plausible-sounding fiction.

What still failed:

- No few-shot example of what strong reasoning looks like
- Design decisions present but surface-level compared to what they could be
- No defined output structure — sections appeared but in inconsistent order
- No curl examples yet

What I'd try next: Add a few-shot example to show what genuinely specific design reasoning looks like.

V3 — Few-Shot Examples

Technique: Few-Shot Examples

Prompt addition:

Here is an example of a good "Design Decisions" section from a different project. Match this level of specificity and reasoning:

Example:

"Why Redis over PostgreSQL: The session data has a TTL of 15 minutes and does not need persistence across restarts. Redis handles expiry natively. PostgreSQL would require a cron job to clean expired rows — unnecessary complexity for this use case.

Why JWT over sessions: The API is stateless by design. Sessions require server-side storage, which contradicts the stateless constraint."

Now write the Design Decisions section for the FlyRank Todo API at this level of specificity.

Output (Representative Excerpt):

Why an in-memory dict over SQLite (even though SQLite needs no separate server): The assignment's objective was to test REST and HTTP fluency, not persistence-layer design. Even file-based SQLite pulls in schema setup, connection lifecycle handling, and row/object serialization concerns orthogonal to what was being evaluated. A dict keyed by integer task ID gives $O(1)$ lookups.

Why PUT for partial updates instead of PATCH: Implementing PATCH correctly per RFC 6902 (JSON Patch) or RFC 7396 (JSON Merge Patch) would have added a spec-compliance layer disproportionate to the assignment's scope. Using PUT with an "all fields optional" schema is a conscious deviation from textbook REST — documented here rather than left implicit.

Why no locking around the in-memory store: Uvicorn's default worker runs single-process on the asyncio event loop. No route handler awaits mid-mutation. A lock would be inert in this execution model, so one wasn't added just to look thorough.

Note (V3)

What changed in the prompt: Added a concrete example of strong design reasoning from a different project.

What improved in the output: The design decisions section became genuinely specific. Each decision now named what was rejected and why. "A lock would be inert in this execution model, so one wasn't added just to look thorough" — that sentence could only describe this project. The RFC references for PATCH showed real technical depth. The few-shot example raised the bar visibly and measurably. The model matched the pattern of the example: named the alternative, explained why it was rejected, stated the tradeoff explicitly.

What still failed:

- Output structure still inconsistent — sections in different order each run
- No curl examples still
- How to Run section generic — assumed FastAPI knowledge

What I'd try next: Define exact output structure so the full README is consistently organized.

V4 — Output Structure

Technique: Output Structure

Prompt addition:

Structure the full README in exactly this order:

1. Project title and one-line description
2. Tech stack (table)
3. Endpoints (table with method, path, status code, description)
4. Design decisions (specific reasoning per decision)
5. How to run (install + start commands)
6. How to test (curl examples for each endpoint)
7. Windows note (PowerShell curl.exe fix)
8. AI Rematch section (what the AI version did differently and why it matters)
9. Key learnings

Output (Representative Excerpt):

FlyRank Todo API

A REST CRUD API for managing tasks, built with Python and FastAPI using in-memory storage only
— no database.

[Tech stack table]

[Endpoints table with status codes]

[Design decisions — specific reasoning]

How to Test

```
curl.exe -X POST http://127.0.0.1:8000/tasks \  
-H "Content-Type: application/json" \  
-d '{"title": "Write README", "priority": "high"}'
```

curl.exe <http://127.0.0.1:8000/tasks>

curl.exe <http://127.0.0.1:8000/tasks?completed=true>

curl.exe -X DELETE <http://127.0.0.1:8000/tasks/1>

curl.exe <http://127.0.0.1:8000/stats>

curl.exe -X POST <http://127.0.0.1:8000/reset>

Note (V4)

What changed in the prompt: Defined the exact section order and section names.

What improved in the output: The README became a complete, consistently structured document for the first time. Every section appeared in the right order across runs. Curl examples appeared for every endpoint. The AI Rematch section appeared and was grounded in real project details. A stranger could clone the repo, follow this README, and test every endpoint without asking a single question. The output stopped being a collection of good sections and became a coherent document.

What still failed: How to Run still assumed FastAPI knowledge. The steps were not explicit enough for someone who had never used uvicorn before.

What I'd try next: Add step decomposition to make How to Run explicit for a FastAPI newcomer.

V5 — Step Decomposition

Technique: Step Decomposition

Prompt addition:

For the "How to Run" section, break it into explicit numbered steps. Assume the reader knows Python but has never used FastAPI or uvicorn before. Each step should be one action with one command.

Output (Representative Excerpt):

How to Run

Assumes Python is installed and you've never used FastAPI or uvicorn before. Run each command from the project root.

1. Create a virtual environment:

```
python -m venv venv
```


2. Activate the virtual environment:

- Windows: `venv\Scripts\activate`
- macOS/Linux: `source venv/bin/activate`

3. Install the dependencies:

```
pip install fastapi uvicorn
```

4. Start the server:

```
uvicorn main:app --reload
```

5. Confirm it's running:

```
http://127.0.0.1:8000/health
```

```
Expected: {"status": "ok"}
```

6. Open Swagger UI:

```
http://127.0.0.1:8000/docs
```

The `--reload` flag restarts the server automatically whenever you save a code change.

Note (V5)

What changed in the prompt: Required explicit numbered steps, one action per step, targeted at someone who knows Python but not FastAPI.

What improved in the output: How to Run became usable by a stranger. Windows and macOS commands were separated. The health check step confirmed the server was running before testing began. Swagger UI was pointed to explicitly. The `--reload` flag was explained rather than

just shown. Previously this section transferred nothing — it assumed knowledge. Now it transfers it. This is the first version a non-FastAPI developer could follow end-to-end without help.

What still failed: Step decomposition only applied to How to Run. The curl examples section could use the same treatment — grouping by scenario rather than just listing commands.

Cross-Model Comparison

Final Prompt (sent to both models):

You are a senior backend engineer writing a GitHub README for your own internship project.

Project:

- Python + FastAPI
- In-memory storage only — intentional constraint
- Full REST CRUD: GET, POST, PUT (partial), DELETE
- HTTP status codes: 200, 201, 204, 404, 422
- Pydantic v2 validation
- Endpoints: /tasks, /tasks/{id}, /stats, /reset, /health
- Query filtering: ?done=true and ?search=
- Seeded with 3 tasks on startup
- Tested on Windows using curl.exe
- PowerShell fix: write body to file, use -d "@body.json"
- AI rematch: ran AI version on port 8001 simultaneously
- 3 differences found: AI required title in PUT, converted 422 to 400, started with empty list

Structure the README in exactly this order:

1. Title and one-line description
2. Tech stack (table)
3. Endpoints (table: method, path, status, description)

4. Design decisions (specific reasoning per decision)
5. How to run (numbered steps, one action each, reader knows Python but not FastAPI)
6. How to test (curl examples per endpoint)
7. Windows note
8. AI Rematch section
9. Key learnings

For design decisions, match this level of specificity: "Why Redis over PostgreSQL: The session data has a TTL of 15 minutes. Redis handles expiry natively. PostgreSQL would require a cron job — unnecessary."

Tone: direct, plainspoken. No buzzwords. No "robust", "scalable", "seamless". Developer voice.

Claude — What Actually Happened

Tone: Matched the voice instruction precisely. No buzzwords appeared. Design decisions used the named-alternative pattern from the few-shot example consistently. Phrases like "a lock would be inert in this execution model, so one wasn't added just to look thorough" showed the model writing as someone accountable for their choices.

Structure: Followed the exact section order. Tables were clean and accurate. Curl examples used curl.exe correctly throughout.

Accuracy: Design decisions grounded in real technical reasoning — referenced RFC 6902 and RFC 7396 for the PUT vs PATCH decision. AI Rematch section identified specific differences and framed them correctly: the differences revealed prompt ambiguity, not AI unreliability.

Failure point: Key learnings section was the weakest part. Learnings listed as bullet points but not always tied to a specific decision made during the project. Felt slightly generic compared to the rest of the document.

ChatGPT — What Actually Happened

Tone: Structurally compliant but tone drifted. Despite the explicit instruction against buzzwords, the output framed the AI Rematch section as:

"AI can significantly accelerate backend development"

That is the kind of sentence the voice card was designed to prevent. It sounds like a LinkedIn post, not a developer writing about their own project.

Structure: Followed section order correctly. Tables present. However, curl examples used ^ for line continuation (Windows CMD style) rather than the backtick style used in PowerShell — a small but telling detail that showed less environmental awareness.

Accuracy: The AI Rematch section was the most revealing difference. ChatGPT framed it as a comparison between manual and AI implementation approaches in a table:

Manual Implementation | AI-Generated Version

Built incrementally | Generated in a single pass

Explicit decisions | Focused on working code

This framing misses the actual point entirely.

The real lesson from the AI rematch was: My prompt was vague. The AI made architectural decisions I never specified. ChatGPT turned it into a generic "human vs AI" comparison that could describe any project. Claude identified the specific architectural decisions that differed and why.

Failure point:

Key learnings were listed as a series of generic observations about FastAPI and Pydantic. They did not connect to specific decisions made during this project. A reader could not tell these learnings came from this project specifically.

Honest Comparison

Dimension	Claude	ChatGPT
Voice card compliance	✓ No buzzwords	✗ LinkedIn phrases appeared
Windows context	✓ curl.exe correct	⚠ CMD-style line continuation
AI Rematch framing	✓ Named specific differences	✗ Generic human vs AI table
Design reasoning	✓ Named alternatives rejected	✓ Named alternatives rejected
Section order	✓ Exact	✓ Exact

Key learnings	⚠ Slightly generic	✗ Generic, not project-specific
---------------	--------------------	---------------------------------

Summary:

Both models produced structurally correct READMEs. The difference was in framing and voice precision. Claude respected the voice constraint more strictly. ChatGPT drifted on tone despite the explicit instruction. The AI Rematch section was the sharpest difference — Claude identified what actually happened, and ChatGPT produced a generic observation that could describe any project.

The most important finding: the same prompt produced meaningfully different framing on the most important section of the document. Voice constraints need to be stricter and more specific for ChatGPT than for Claude to get equivalent results.

Final Reusable Template

You are a senior backend engineer writing a GitHub README for your own project.

Project details:

[FILL: language and framework]

[FILL: storage type and the reason it was chosen]

[FILL: list every endpoint with its method]

[FILL: HTTP status codes used and which endpoints use them]

[FILL: validation library]

[FILL: OS-specific testing notes]

[FILL: any stretch features beyond core requirements]

[FILL: any experiment or comparison conducted]

Structure the README in exactly this order:

1. Title and one-line description
2. Tech stack (table: component, choice, why)

3. Endpoints (table: method, path, status codes, description)

4. Design decisions — for each decision:

- Name what was chosen
- Name what was rejected
- Explain why in one to three sentences
- Example pattern: "Why X over Y: [reason Y was rejected], [reason X was chosen], [tradeoff acknowledged]"

5. How to run:

- Numbered steps
- One action per step
- One command per step
- State the assumed knowledge level of the reader

6. How to test:

- One curl example per endpoint
- Include at least one error case (404 and 422)

7. Platform note (any OS or tooling-specific fix)

8. Experiment or comparison section:

- Name the specific differences found
- Explain what each difference reveals
- Do not frame as generic "AI vs human" comparison

9. Key learnings:

- Each learning tied to a specific decision in this project
- Not transferable to any other project without editing

Tone rules (enforce strictly):

- No buzzwords: "robust", "scalable", "seamless", "cutting-edge", "leverage", "streamline"

- Write as a developer, not a marketer
- Every design decision names what was rejected and why
- If a learning could describe any project, cut it

What Each Technique Actually Did

Version	Technique	Real Output Change
V0	None	Structurally reasonable but invented project details — wrong priority field, assumed folder structure
V1	Role Assignment	Tone became developer-like, reasoning appeared — but still invented details without real context
V2	Real Context	Biggest jump. Invented details disappeared. Endpoint table matched real project. Windows note appeared and was accurate
V3	Few-Shot Examples	Design decisions went from surface-level to specific. Model started naming what was rejected and why
V4	Output Structure	Full document appeared consistently. Curl examples appeared. AI Rematch section grounded in real project
V5	Step Decomposition	How to Run became usable by a stranger who has never used FastAPI

One Honest Failure Moment

V1 did not solve the core problem.

Role assignment was supposed to be a meaningful early lever. It wasn't. The model wrote more confidently and with better reasoning — but it was still confidently wrong. It invented a priority field and assumed a folder structure that doesn't exist in my project. Confidence without accurate information is worse than uncertainty with accurate information. A confident wrong README sends someone in the wrong direction. An uncertain correct one at least points them somewhere real. Role assignment improved tone. It did nothing for accuracy. Real context (V2) fixed accuracy in one step. That was the actual lever. Role was secondary. If I ran this ladder again, I would swap the order — context before role. Context is what the model needs to be useful. Role is what shapes how it sounds once it has something real to say.